

Authorized Push Payment Fraud and Data Analytics

Micah Holmquist

BUS 598 Business Analytics Project (Directed Study)

Dr. Jaeseung Baek

March 27, 2026

Table of Contents

1. Introduction	4
2. Data Description	8
3. Pre-Analysis	13
4. Methods	23
5. Analysis of Decision Tree	26
6. Analysis of Random Forests	28
7. Analysis of K-Nearest Neighbors (KNN)	31
8. Comparing All Three Models on when Run on Testing Data	33
9. Looking at the Superior Model	34
10. Business and Managerial Implications	39
11. Conclusion	42
12. References	43

List of Figures

1. Non-Fraudulent and Fraudulent Split in Condensed Data	13
2. Summary Statistics for Income Variable	13
3. Customer Age Counts	14
4. Payment Type Bar Chart	16
5. Employment Status Bar Chart	19
6. Housing Status Bar Chart	20
7. Foreign Request Bar Chart	21
8. Source Bar Chart	21
9. Month Bar Chart	23
10. Decision Tree	26
11. Confusion Matrix	27
12. Random Forest (600-Tree Model) Confusion Matrix – Validation	29
13. Accuracy and AUC for Each k	31
14. KNN (k = 51) Confusion Matrix – Validation	32

List of Tables

1. Variables and Their Description	9
2. Customer Age Descriptive Statistics	14
3. Descriptive Statistics for Days Since Request	15
4. Descriptive Statistics for Intended Balcon Amount	16
5. Descriptive Statistics for Velocity (6h)	17
6. Descriptive Statistics for Velocity (24h)	18
7. Descriptive Statistics for Velocity (4w)	18
8. Descriptive Statistics for Proposed Credit Limit	20
9. Descriptive Statistics for Session Length in Minutes	22
10. Decision Tree Validation Set: Accuracy and Statistical Significance	28
11. Random Forest Validation Accuracy, Error, and AUC by Number of Trees	29
12. Random Forest (600 Trees) Validation Set: Accuracy and Statistical Significance	30

13. KNN ($k = 51$) Validation Set: Accuracy and Statistical Significance	33
14. Test Set Model Comparison: Accuracy and AUC	33
15. Random Forest Model Variable Importance	34

Authorized Push Payment Fraud and Data Analytics

1. Introduction

Fraudulent financial transactions are a major concern for banks, individuals, and institutions. A Nasdaq report estimated that nearly \$500 billion was lost in financial fraud in 2023 (2024). This only measures first-order impacts while Bao has theorized that financial fraud erodes public trust and can hinder economic growth both because the decreased trust discourages investment and because there are costs associated with regulations implemented by governments to discourage financial fraud (2023). On a micro level, any financial institution that is able to decrease the number of fraudulent transactions it processes is likely to inspire more trust from consumers.

Given this situation, it is understandable that financial institutions would be interested in ways to accurately identify fraudulent financial transaction requests before they have been completed and machine learning has emerged as a method for accomplishing this goal (Galoustian, 2025; Yang, 2025).

One form of financial fraud is authorized push payment (APP) fraud. Unlike traditional cybercrime, which relies on technological compromises such as credential theft, malware, or exploitation of technological vulnerabilities, APP fraud is when someone is deceived by social engineering into sending money directly from their own bank account thinking that they are engaging in a legitimate transaction (Braithwaite, 2024; Lalchand et al., 2025; Ngan, 2025; Shum, 2025).

Authorized push payment (APP) fraud is not an abstract problem; it is an operational challenge for banks, credit unions, fintech firms, and digital payment platforms that provide online and mobile transfers. These organizations are expected to move money quickly and

conveniently, but faster payment systems reduce the time available to detect and stop suspicious transfers once a customer initiates them (Braithwaite, 2024; Lalchand et al., 2025). APP fraud is also difficult to identify because the payment is authorized by the account holder. This means it can look “legitimate” in terms of login credentials and normal account behavior even when the customer is being manipulated through social engineering (Braithwaite, 2024; Ngan, 2025). For financial institutions, missed APP fraud can create direct losses, investigation costs, and reputational damage, while overly aggressive controls can increase false alarms that delay legitimate payments and raise the workload for fraud operations teams (Lalchand et al., 2025; Vona, 2017). Because of these tradeoffs, effective detection is important not only to reduce fraud losses, but also to protect customer trust and keep digital payment systems running efficiently (Lalchand et al., 2025; Vona, 2017).

APP fraud is a growing problem. Lalchand et al. (2025) estimate that APP fraud in the United States totaled \$8.3 billion in 2024 and project it will grow to \$14.8 billion by 2028. Researchers attribute the growth of APP fraud to broader economic and technological developments. Braithwaite (2024) and Ngan (2025) posit that the desire for faster payments and mobile banking as a means of facilitating online commerce has increased opportunities for fraud. Faster payment systems reduce the time available for detection and intervention, making prevention more difficult. Lalchand et al. (2025) further suggests that advances in artificial intelligence (AI) have enhanced the sophistication and scalability of fraud schemes, enabling fraudsters to generate more convincing communications and that the continued growth of AI will only exacerbate this trend.

The consequences of financial fraud, including APP fraud, go beyond the previously mentioned financial losses. Ngan (2025) stresses that victims often feel emotionally harmed

while a variety of scholars posit that the occurrence of financial fraud, including APP fraud, fuels a broader erosion of trust in financial institutions and payment systems (Arrow, 1972; Knack & Keefer, 1997; Pavlou, 2003; Sapienza & Zingales, 2012). On the macroeconomic level, this lessening of trust can potentially discourage investment activity (Bao, 2023), meaning that, even just in terms of financial damage, the consequences of this fraud go far beyond the actual amount of fraud. On the individual level, Vona (2017) makes the point that consumer perceptions of how financial institutions handle financial fraud can influence the financial institutions that they choose to use. This suggests that fraud prevention is increasingly important to financial institutions in terms of maintaining not just customer trust but also their competitiveness in the market.

Although combatting APP fraud is therefore important to financial institutions, fraud detection methods have tended to focus on traditional methods of fraud, this change represents a new challenge. While things like an unexpected country-of-origin or the same device being used to initiate financial transactions for multiple accounts are helpful in identifying traditional financial fraud, they are not helpful in identifying APP fraud where the owner of the account will use their own device in a place they regularly frequent to initiate the transaction (Braithwaite, 2024).

Financial institutions are already responding to authorized push payment (APP) fraud by prevention controls and formalizing customer reimbursement processes. This is particularly true in environments where regulation has increased the operational and financial consequences of scam losses. In the United Kingdom, for example, the Payment Systems Regulator implemented mandatory reimbursement requirements for APP scams in 2024, which increases the incentive for payment firms to stop suspicious transfers before money leaves the account and to improve

how scam cases are handled when they occur (Payment Systems Regulator, 2026). Public reporting and commentary on these rules additionally indicates that banks have been updating and improving their fraud controls and warning/intervention processes in response to the reimbursement framework (Shaw, 2024). At the individual firm level, some banks have also published reimbursement policies that describe how APP scam claims are assessed and processed, reflecting that APP fraud has become not only a fraud-loss issue but also an operational service and compliance issue (Standard Chartered, n.d.).

The goal of this paper is to look at how a financial institution could better identify cases of potential APP fraud. It will do this by using machine learning techniques to create predictive models that its employees could use to better identify cases of potential APP fraud. If an institution were to improve its effectiveness at identifying cases of potential APP fraud, it would be able to reduce the amount of money lost to APP fraud and inspire greater confidence in its customers and potential customers. Any model that this process creates would, in order to be effective, be something that a lay person -i.e. an employee who does not know all of the technical information behind the creation of the model- would be able to use and so any model created will be one that is easy to understand and has a binary output; the model will tell the user that a transaction is or is not likely enough to be potentially fraudulent that an investigation is warranted. While identifying cases of potential APP fraud is important, allowing transactions that are unlikely to be fraudulent to go through with minimal delay is also important for making customers happy.

This paper will include a discussion of the data being used followed by analyzing the data with descriptive statistics. Then it will discuss the machine learning techniques used to create the models with this being followed by a look at the actual use of these techniques to create models

in this specific scenario. Finally, the models will be evaluated as their effectiveness, the elements that make up the most effective model will be analyzed, and, most importantly, this paper will conclude with a discussion of the implications of these findings for financial institutions and their employees as they attempt to effectively identify transactions that are more likely to involve APP fraud than others.

2. Data Description

The dataset selected from this project (Jesus et al., 2022) comes from kaggle.com and combines six synthetically created datasets that were all created for the purpose of using machine learning to detect fraudulent financial transactions. Synthetically created data is not produced by real world events, but rather has been artificially created, often the purposes of using machine learning techniques (Hashemi-Pour et al., 2024). Synthetic data is popular for this purpose because the use of real world data is generally regarded as riskier in terms of protecting private user data (Hashemi-Pour et al., 2024; Reiter, 2005; Stadler et al., 2022) and thus synthetic data is safer to be shared and used widely (Raghunathan et al., 2003).

The disadvantage to synthetic data is that because it is only modeled after real world data, it is possible that the modeling is not perfectly accurate, which could undermine the accuracy of any model that the data is used to create (Snoke et al., 2018). This is especially important to keep in mind with the data use here (Jesus et al., 2022) as it was created for the purposes of identifying possible cases of financial fraud, but not specifically APP fraud. Still, the process of using data with data mining techniques is the same and so the paper will still show the correct process.

This dataset that will be used has one million observations of synthetically created transactions with 31 independent variables. Of the one million observations, 11,029 or roughly 1.1% are labeled as fraudulent. There were no missing variables (Jesus et al., 2022).

The variables are listed and described in Table 1.

Table 1: and their description.

Variable	Description	Type of Variable
fraud_bool	A binary variable that rates whether a transaction is fraudulent or not. The options are 0/1, which equate to non-fraudulent/fraudulent. This is the dependent/output variable while all other variables are independent.	Categorical
Income	A measurement of the purported financial transactor's income in quantiles. The variable is continuous and bounded on the interval 0,1.	Numerical
name_email_similarity	A measurement of how similar the purported financial transactor's name is to the email given. The variable is continuous and bounded on the interval 0,1.	Numerical
prev_address_months_count	Number of months the purported financial transactor has had a previous address on record.	Numerical
current_address_months_count	Number of months the purported financial transactor has had their current address.	Numerical
customer_age	The age in years of the purported financial transactor.	Numerical
days_since_request	Number of days since the financial transaction was initiated.	Numerical
intended_balcon_amount	Intended amount to transfer. The variable is continuous	Numerical

	and bounded on the interval - 1,108.	
payment_type	The payment type used. There are five types of payments.	Categorical
zip_count_4w	Number of financial transactions from the same zip code in the last four weeks.	Numerical
velocity_6h	The frequency of activity associated with the transaction in the last six hours.	Numerical
velocity_24h	The frequency of activity associated with the transaction in the last 24 hours.	Numerical
velocity_4w	The frequency of activity associated with the transaction in the last four weeks.	Numerical
bank_branch_count_8w	The number of bank branches associated with the transaction in the last eight weeks.	Numerical
date_of_birth_distinct_emails_4w	The number of different email addresses grouped by the given data of birth in the last four weeks.	Numerical
employment_status	The employment status of the purported financial transactor. Limited to nine variables.	Categorical
credit_risk_score	The credit risk score of the purported financial transactor.	Numerical
email_is_free	A binary variable based on whether the email address provided comes from a free email service such Gmail, Yahoo Mail, etc.	Categorical
housing_status	The housing status of the purported financial transactor. Limited to nine variables.	Categorical

phone_home_valid	A binary variable based on whether the given home phone number is valid.	Categorical
phone_mobile_valid	A binary variable based on whether the given mobile phone number is valid.	Categorical
bank_months_count	The number of months the purported financial transactor has had an account with the bank.	Numerical
has_other_cards	A binary variable based on whether the purported financial transactor has any credit or debit cards in addition to the one being used in the transaction.	Categorical
proposed_credit_limit	The credit limit of the purported financial transactor prior to the transaction in question.	Numerical
foreign_request	A binary variable based on whether the country where the transaction is initiated is different from the country where the financial institution is located.	Categorical
Source	The venue in which the financial transaction was initiated. Limited to four variables.	Categorical
session_length_in_minutes	The number of minutes the session where the financial transaction was requested lasted.	Numerical
device_os	The operating system of the device used to request the financial transaction.	Categorical
keep_alive_session	A binary variable on whether on session where the financial transaction was requested was kept alive.	Categorical
device_distinct_emails_8w	The number of unique email addresses known to be used on the device where the financial transaction was	Numerical

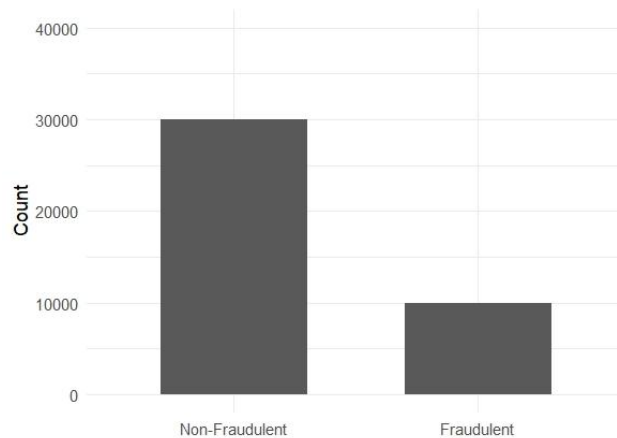
	requested in the last eight weeks.	
device_fraud_count	The number of confirmed or suspect fraud cases on the device used to request the financial transaction.	Numerical
Month	The month in which the financial transaction was requested. The dataset includes an 8-period monthly time index; the variable month is encoded from 0 to 7, representing eight consecutive months (relative month index) within the dataset window.	Categorical

Source: Jesus et al., 2022.

The values for name_email_similarity, previous_address_months_count, current_address_months_count, zip_count_4w, bank_branch_count_8w, date_of_birth_distinct_emails_4w, email_is_free, phone_home_valid, phone_mobile_valid, bank_months_count, has_other_cards, device_os, keep_alive_session, device_distinct_emails_8w, and device_fraud_count were removed as they are not relevant to APP fraud due to the nature of this fraud. To use an example, if someone is tricked into sending money, they would still be expected to be doing so from their device (device_fraud_count) and to be able to enter their phone number correctly (phone_mobile_valid).

The data was randomly reduced to 40,000 observations with a desired 30/10 split between fraudulent and non-fraudulent transactions as seen in Figure 1.

Figure 1: Non-Fraudulent and Fraudulent Split in Condensed Data.



3. Pre-Analysis

As noted above, the income variable is based on categories ranging from 0.1 to 0.9. The details of the entries for this variable can be seen in Figure 2.

Figure 2: Summary Statistics for income variable.

Mean	0.59
Standard Error	0.001
Median	0.70
Mode	0.90
Standard Deviation	0.29
Sample Variance	0.08
Kurtosis	-1.18
Skewness	-0.52
Range	0.80
Minimum	0.10
Maximum	0.90
Count	40000

The median value of 0.7 means that half of the values are 0.7 or greater. This is consistent with the skewness score of -0.52, which indicates the results are left-skewed. Dispersion is moderate (standard deviation = 0.29). The -1.18 Kurtosis score indicates that the distribution is

flatter than a normal distribution. All of this indicates that the higher incomes are represented at greater levels than lower incomes.

Figure 3: customer_age Counts.

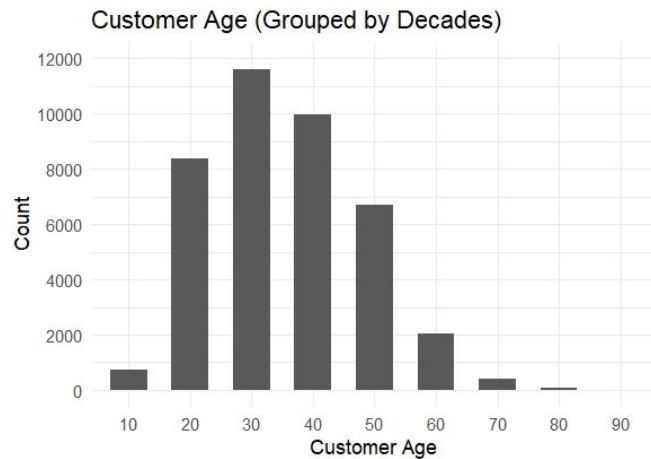


Table 2: customer_age Descriptive Statistics

Mean	0.59
Standard Error	0.001
Median	0.70
Mode	0.90
Standard Deviation	0.29
Sample Variance	0.08
Kurtosis	-1.18
Skewness	-0.52
Range	0.80
Minimum	0.10
Maximum	0.90
Count	40000

As seen in Figure 3, the customer_age variable records age in 10-year age bands (decades) rather than exact ages. Specifically, ages were coded into the categories 10, 20, 30, ..., 90, where each value represents the corresponding decade group (e.g., 20 = ages 20–29, 30 = ages 30–39).

As seen in Table 2, the customer_age variable had a mean of 0.59 and media of 0.70, indicating that values were typically towards the higher end of this age scale. Dispersion was

moderate (standard deviation = 0.29, variance = 0.08). The distribution was negatively skewed (skewness = -0.52), meaning more observations were clustered at higher values with a longer tail toward lower values, and it was relatively flat/light-tailed compared to a normal distribution (kurtosis = -1.18).

Table 3: Descriptive statistics for days_since_request

Mean	1.01
Standard Error	0.03
Median	0.01
Mode	0.0001
Standard Deviation	5.40
Sample Variance	29.21
Kurtosis	108.22
Skewness	9.41
Range	75.50
Minimum	0.00
Maximum	75.50
Count	40000

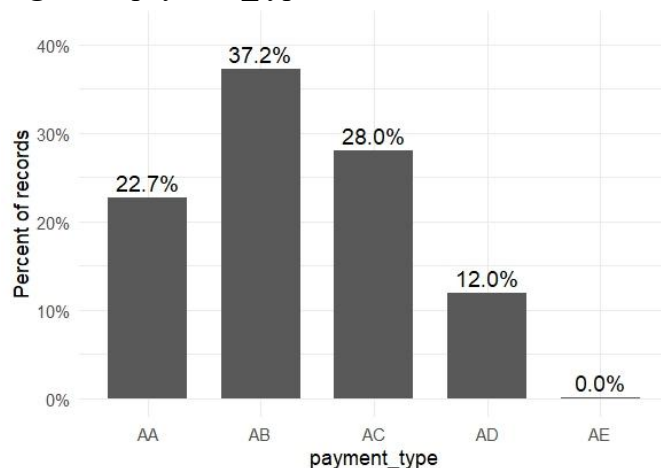
The variable days_since_request has, as indicated in Table 3, a mean of 1.01 days, but a median of 0.01 days and a mode of just above 0. This indicates that most requests occurred very shortly after the person's most recent request. Variability was high (standard deviation = 5.40; variance = 29.21), with values ranging from approximately 0.00 to 75.50 days. The distribution was extremely right-skewed (skewness = 9.41) and highly leptokurtic (kurtosis = 108.22), consistent with a heavy-tailed pattern in which a small number of observations at long delays substantially increase the mean relative to the median.

Table 4: Descriptive Statistics for intended_balcon_amount.

Mean	7.55
Standard Error	0.10
Median	-0.85
Mode	-0.26
Standard Deviation	19.64
Sample Variance	385.62
Kurtosis	8.54
Skewness	2.80
Range	122.43
Minimum	-11.11
Maximum	111.32
Count	40000

The descriptive statistics for variable intended_balcon_amount are in Table 4. The variable has a mean of 7.55, but a median of -0.85 and a mode of -0.26), indicating that typical values were slightly negative while a subset of large positive values increased the average. Variability is substantial (standard deviation = 19.64; variance = 385.62), with values ranging from -11.11 to 111.32 (range = 122.43). The distribution is strongly right-skewed (skewness = 2.80) and leptokurtic (kurtosis = 8.54), consistent with a heavy-tailed distribution containing influential high-end outliers.

Figure 4: payment_type Bar Chart.



The payment_type variable is, as shown in Figure 4, made up of five possible values, with the highest percentage belonging to AB (37.2%) followed by AC (28%), AA (22.7%), and then AD (12%). The payment type was AE for nearly zero percent of the transactions.

Table 5: Descriptive Statistics for velocity_6h.

Mean	5562.89
Standard Error	14.93
Median	5243.76
Mode	4389.11
Standard Deviation	2985.17
Sample Variance	8911266.16
Kurtosis	0.01
Skewness	0.56
Range	16777.24
Minimum	-170.60
Maximum	16606.64
Count	40000.00

The variable velocity_6h has, as seen in Table 5, a mean of 5,562.89, a median of 5,243.76, and a mode of 4,389.11. This indicates a typical value is in the low-to-mid 5,000s. Variability was substantial (standard deviation = 2,985.17; variance = 8,911,266.16), with values spanning from -170.60 to 16,606.64 (range = 16,777.24). The distribution showed mild positive skew (skewness = 0.56) and approximately normal tail weight (kurtosis = 0.01), suggesting a near normal distribution but with a few higher values stretching it slightly to the right.

Table 6: Descriptive Statistics for velocity_24h.

Mean	4727.48
Standard Error	7.34
Median	4722.87
Mode	5184.53
Standard Deviation	1468.28
Sample Variance	2155836.07
Kurtosis	-0.38
Skewness	0.31
Range	8048.77
Minimum	1366.48
Maximum	9415.24
Count	40000

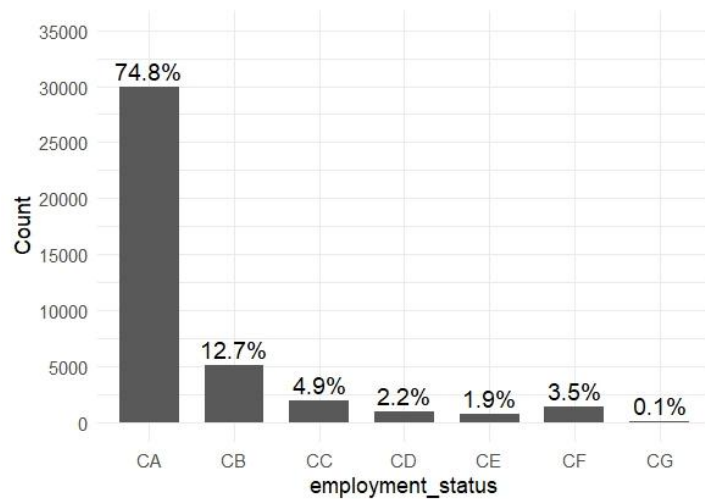
The variable velocity_24h has, as seen in Table 6, a mean of 4,727.48 and a median of 4,722.87, which indicates that the central tendency of activity in the last 24 hours is around 4,700. Variability was moderate (standard deviation = 1,468.28; variance = 2,155,836.07), with values ranging from 1,366.48 to 9,415.24 (range = 8,048.77). The distribution was slightly right-skewed (skewness = 0.31) and showed modestly negative kurtosis (kurtosis = -0.38), consistent with a somewhat flatter-than-normal distribution with a mildly longer right tail.

Table 7: Descriptive Statistics for velocity_4w.

Mean	4833.60
Standard Error	4.70
Median	4890.24
Mode	5626.43
Standard Deviation	939.10
Sample Variance	881900.57
Kurtosis	-0.42
Skewness	-0.02
Range	4070.68
Minimum	2863.78
Maximum	6934.47
Count	40000

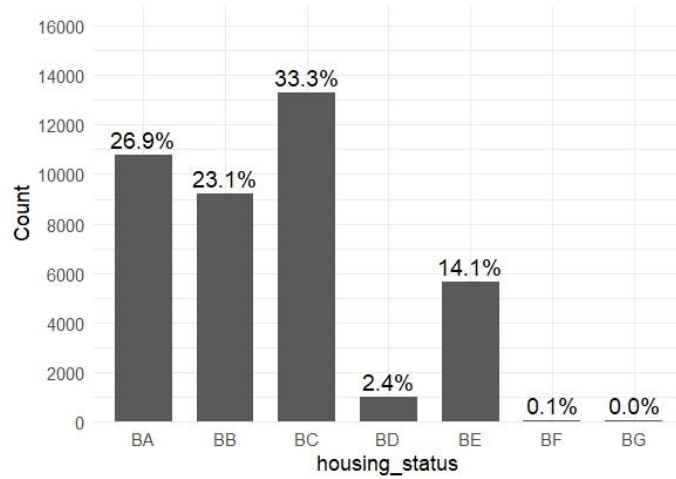
The variable `velocity_4w` has, as seen in Table 7, a mean of 4,833.60 and a median of 4,890.24. These results indicate a central value around 4,800–4,900. Dispersion is relatively low compared with the shorter windows (standard deviation = 939.10; variance = 881,900.57), with values ranging from 2,863.78 to 6,934.47 (range = 4,070.68). The distribution is approximately symmetric (skewness = -0.02) with slightly negative kurtosis (kurtosis = -0.42), suggesting a flatter-than-normal shape with no meaningful skew.

Figure 5: `employment_status` Bar Chart.



The most common value for the `employment_status` variable is, as indicated in Figure 5, CA (74.8%) and followed by CB (12.7%), CC (4.9%), CF (3.5%), CD (2.2%), CE (1.9%), and CG (0.1%).

Figure 6: housing_status Bar Chart.



The most common value for housing_status is, as illustrated in Figure 6, BC (33.33%), followed by BA (26.9%), BB (23.1%), BE (14.1%), BD (2.4%), and BF (0.1%). Virtually none of the values were BG.

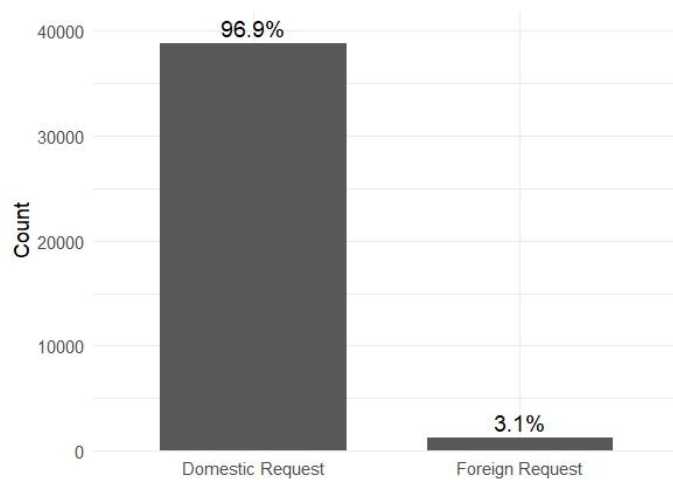
Table 8: Descriptive Statistics for proposed_credit_limit.

Mean	594.06
Standard Error	2.73
Median	200.00
Mode	200
Standard Deviation	546.74
Sample Variance	298925.02
Kurtosis	-0.44
Skewness	1.05
Range	1910.00
Minimum	190.00
Maximum	2100.00
Count	40000

The proposed_credit_limit variable, as seen in Table 8, has a mean of 594.06 and a median and mode both equal to 200, indicating that typical proposed limits are clustered at 200 while higher limits increased the average. Dispersion is substantial (standard deviation

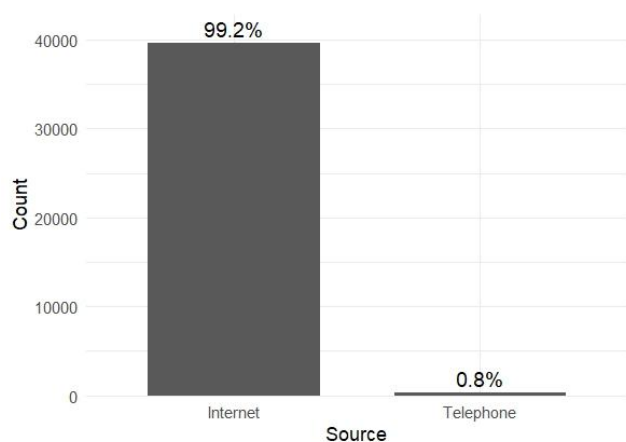
= 546.74; variance = 298,925.02), with values ranging from 190 to 2,100 (range = 1,910). The distribution is right-skewed (skewness = 1.05) with slightly negative kurtosis (kurtosis = -0.44), consistent with a long right tail but a somewhat flatter-than-normal overall shape.

Figure 7; foreign_request Bar Chart.



As the bar chart in Figure 8 makes clear, foreign requests make up 3.1% of the values while the remaining 96.9% are not domestic requests.

Figure 8: source Bar Chart



The variable source produces another set of lopsided results, as seen in Figure 8. Here, 99.2% of requests were made via the internet while the remaining 0.8% were made by telephone.

Table 9: Descriptive Statistics for session_length_in_minutes.

Mean	7.73
Standard Error	0.04
Median	5.13
Mode	3.51
Standard Deviation	8.42
Sample Variance	70.93
Kurtosis	13.86
Skewness	3.26
Range	77.71
Minimum	0.02
Maximum	77.73
Count	39923

For the session_length_in_minutes variable there are 77 observations (0.19%) which had a value of -1 . Since it is unclear how a session could last a negative amount of time and -1 is the only negative value observed, this appears to be a coded value indicating an unknown/invalid session duration rather than a true negative time. These values were thus treated as missing.

Across the remaining 39,923 observations, as Table 9 shows, the session_length_in_minutes variable had a mean of 7.73, a median of 5.13 minutes, and a mode of 3.51 minutes, as can be seen in Table 8. This indicates that most sessions were relatively short but longer sessions increased the average. Dispersion was substantial (standard deviation = 8.42; variance = 70.93), with session lengths ranging from 0.02 to 77.73 minutes (range = 77.71). The distribution was strongly right-skewed (skewness = 3.26) and highly leptokurtic (kurtosis = 13.86), consistent with a heavy right tail driven by a small number of unusually long sessions.

Figure 9: month Bar Chart.

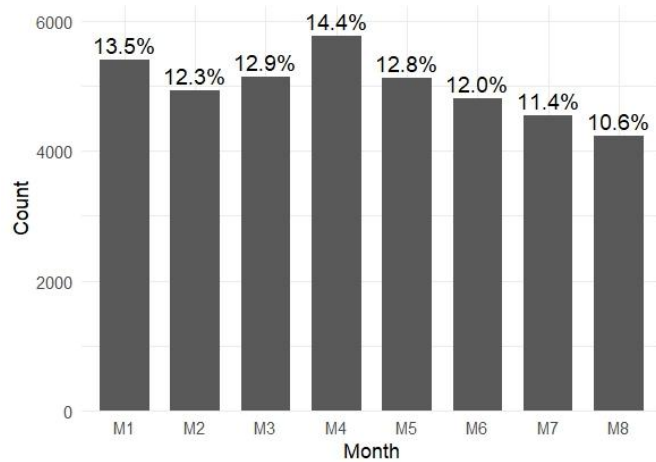


Figure 9 shows that observations are fairly evenly distributed across the eight-month categories (0–7). Under a perfectly uniform distribution, each month would account for 12.5% of observations; in this dataset, the observed monthly shares are close to that benchmark, ranging from 10.6% to 14.4%, indicating only modest deviation from uniformity and no strong temporal concentration in any single month.

4. Methods

Decision trees, random forests, and K-Nearest Neighbors (KNN) are all standard machine learning techniques that this paper will use in order to classify transactions as being either “likely to involve fraud” or “unlikely to involve fraud”. If the former is the case, it does not mean the transaction is fraudulent or even that the model is saying it is fraudulent, but rather that additional attention needs to be paid to the transaction to determine if it is fraudulent (Vona, 2017).

All three methods require that data be separated and so the 40,000 observations were randomly divided into training (80% of the data), validation (10%), and testing (10%) data while still maintaining a three-to-one ratio of non-fraudulent to possibly fraudulent in all three sub-sections. The purpose of this tripart separation is to ensure that the model does not get created

based on the same data that is used to evaluate the final performance of the model (Arlot & Celisse, 2010).

The training data is the portion used to *teach* each model by showing it many examples of transactions along with their variables and the determination if the transaction was potentially fraudulent or not, so the model can learn what patterns tend to be associated with higher risk transactions. The validation data is then used like a *practice exam*: it helps choose the best version of each method (for example, how simple or complex the model should be) without using the final test data. Finally, the testing data is saved until the very end and serves the purpose of a *final exam* to see how well the chosen models perform on transactions they have never seen before. Using this three-step approach helps ensure the results reflect how the models would work in the real world, rather than just how well they can match the specific data used to build them.

Decision trees work like a flowchart of yes/no questions. The model looks at one piece of information at a time (such as the requested amount or how recent the last request was) and follows a set of “if-then” rules to decide whether a transaction should be flagged for review. A benefit of this approach is that the resulting steps can be easily followed and acted upon by a person without a technical background. The downside to a decision tree is that it can be too sensitive, which is to say small changes in the data can lead the tree to produce different rules and if the tree becomes too detailed, it can start fitting random quirks in the training data rather than patterns that hold up with different data, such as the validation and testing sections. In that situation, the tree may look accurate during training but perform less reliably when used to understand different data (Bertsimas et al., 2016; Halabaku, 2024).

Random forests improve on a single decision tree by using many different decision trees and combining their answers. Instead of relying on one flowchart (which might be thrown off by small quirks in the data), the random forest builds lots of trees using slightly different “slices” of the training data and different mixes of the input variables. Each tree makes its own prediction, and the random forest makes the final decision by taking the overall consensus across all of the trees. This “wisdom of the crowd” approach usually produces a model that is more stable and reliable, because it is less likely to be driven by random chance patterns from the training data and therefore tends to perform better on new, unseen transactions (Bertsimas et al., 2016; Breiman, 2001).

K-Nearest Neighbors (KNN) makes a prediction by comparing a new transaction to past transactions that look most similar. The model chooses a number called k , which tells it how many “closest matches” to use. If most of those similar past transactions were labeled as potential fraud, then the new transaction is also more likely to be flagged; if most were legitimate, it is more likely to be cleared. This approach can work well because it does not rely on a single set of fixed rules and can adapt to complex patterns in the data. However, KNN can struggle when there are many different variables describing each transaction, because it becomes harder to decide what “similar” really means when you are trying to compare cases across many dimensions at once (ScienceDirect, n.d.; Pestov, 2013).

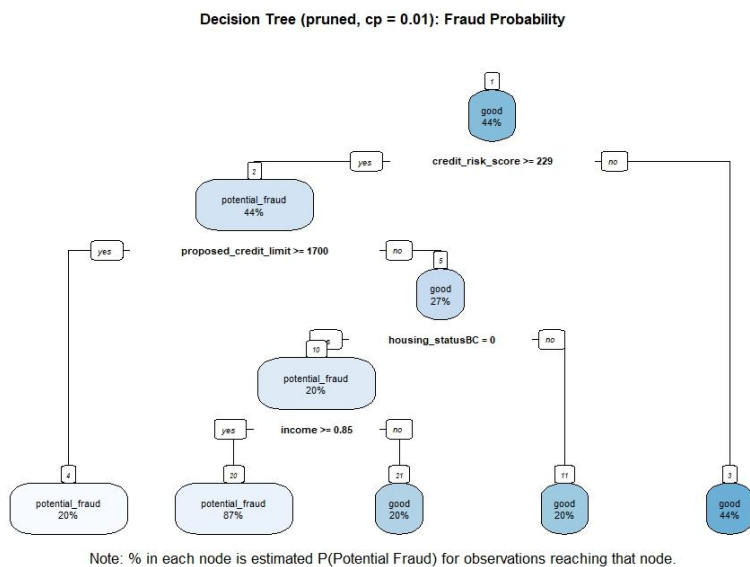
Two metrics were used to summarize test performance:

- Accuracy: the percentage of test transactions the model classified correctly (fraud vs. non-fraud) using a single decision cutoff.
- AUC (Area Under the ROC Curve): a threshold-independent measure of how well the model ranks fraud cases as higher risk than non-fraud cases. An AUC closer to 1 indicates stronger separation; an AUC of 0.5 indicates no better than random ranking.

The use of both the accuracy and AUC methods is helpful because each measures a different aspect of performance. Accuracy reflects how often the model makes the correct prediction at a chosen cutoff, while AUC measures the overall ranking across all possible cutoffs. The model comparison reported in Table 13 below is therefore based on each model's test-set Accuracy and test-set AUC, with the test set kept separate until the end to provide an unbiased estimate of real-world performance.

5. Analysis of Decision Tree

Figure 10: Decision Tree.



Pruning with $cp = 0.01$ is basically a way of editing the decision tree so that it appears readable and simple. A decision tree can keep adding additional splits to better match the training data, but if it is allowed to grow without restraint it may become very large and start fitting random noise rather than meaningful patterns, which can hurt performance on new cases. The complexity parameter (cp) is the key hyperparameter that controls this tradeoff by acting like a minimum “benefit required” for each additional split: if a split does not improve the model enough, it is not worth the added complexity and is removed during pruning.

This is why Figure 10 shows only four variables. Decision trees display the variables that are actually used in the remaining splits, and pruning can remove many splits at once. When those removed branches disappear, any variables that were only used in those cut-off parts of the tree disappear too, leaving only the few predictors that created the most important, high-value splits in the final pruned tree.

Figure 11: Confusion Matrix

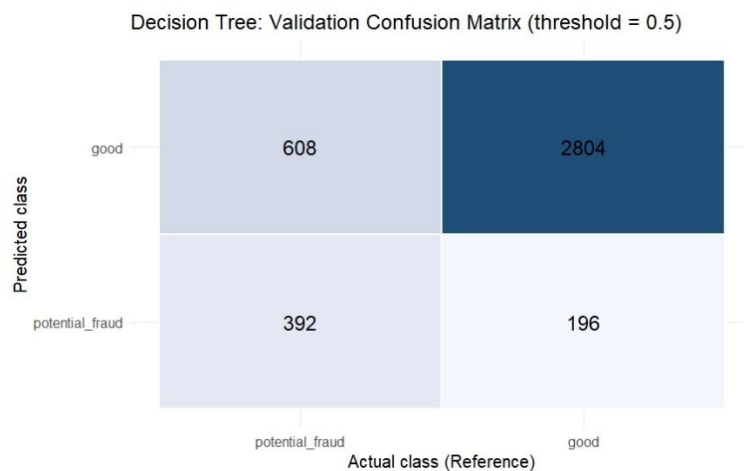


Figure 11 summarizes how well the decision tree performs when it attempts to identify potential cases of fraud. Specifically, it makes the results concrete by showing the actual counts of correct and incorrect classifications. Out of 1,000 fraud cases in the validation set, the model correctly flags 392 as potential fraud (true positives) but misses 608 fraud cases by labeling them “good” (false negatives), which corresponds to a sensitivity of about 39.2%. Out of 3,000 good cases, it correctly clears 2,804 as good (true negatives) and incorrectly flags 196 as potential fraud (false positives). This produces a high specificity of about 93.5%. Since the decision clears most “good” cases correctly while failing to catch a substantial share of the potential fraud cases, the information in the two figures indicates that the decision tree is better at identifying legitimate transactions than detecting cases of potential fraud. Since the goal is to not miss cases of potential fraud, these results are disappointing.

Table 10: Decision Tree Validation Set: Accuracy and Statistical Significance

Metric	Value
Accuracy	0.7568
Accuracy 95% CI Lower	0.7431
Accuracy 95% CI Upper	0.7700
P-Value (Accuracy > NIR)	0.1667

On the validation set, the decision tree model achieved, as reported in Table 10, an accuracy of 0.7568, with a 95% confidence interval ranging over (0.7431, 0.7700). Since the no-information rate (NIR) baseline is 0.75 (predicting a non-suspicious transaction every time), the p-value of 0.1667 indicates that this accuracy is not statistically higher than the baseline at typical significance levels. To put it another way, the decision tree's validation performance is only slightly above what would happen if every transaction were labeled non-suspicious and the small improvement that is seen here could reasonably be due to random variation in the validation sample rather than a clear, reliable gain.

6. Analysis of Random Forests

Table 11: Random Forest Validation Accuracy, Error, and AUC by Number of Trees

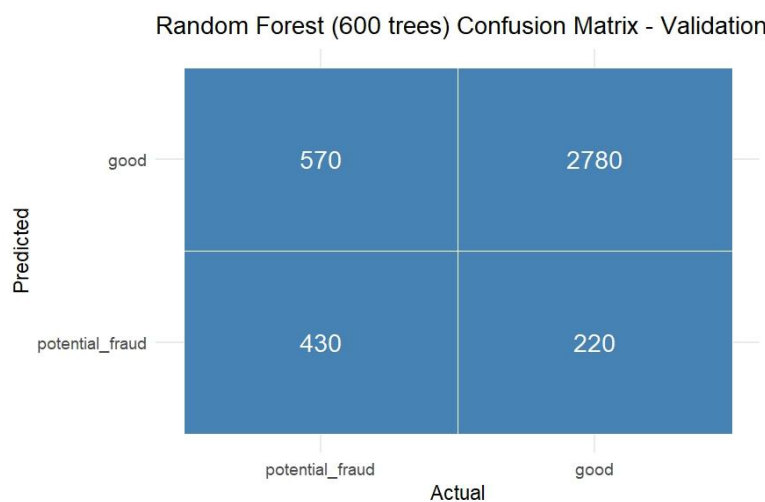
Number of Trees	Accuracy	Error	Accuracy Percentage	AUC
500	0.8028	0.1973	80.28%	0.7982
600	0.8018	0.1983	80.17%	0.8002
700	0.8022	0.1977	80.22%	0.7984
800	0.7998	0.2003	79.97%	0.7970
900	0.8018	0.1983	80.17%	0.7973
1000	0.8032	0.1967	80.33%	0.7983

As shown in Table 11, the random forest's validation performance is highly stable across 500 to 1000 trees. Accuracy stays clustered around 0.80% and the corresponding error rate stays

near 20%. This suggests that accuracy improves very little or not at all when more trees are added. The same can be said when AUC scores are considered. AUC values are tightly grouped between 0.7970 and 0.8002, indicating that the model's ability to better identify cases of potential fraud.

If we consider “best” strictly by the highest AUC, the 600-tree model is the top performer with an AUC of 0.8002, while maintaining essentially the same accuracy (0.8018) as the other models. If we consider “best” strictly by the highest accuracy, the 1000-tree model is marginally highest at 0.8032, but its AUC (0.7983) is not the best and the accuracy improvement over the 600-tree model is only 0.14 percentage points (80.33 percent versus 80.17 percent), which is virtually negligible. in discrimination. If for no other reason than avoiding additional computation, the 600-tree model is marginally better than the 1,000-tree model.

Figure 12: Random Forest 600-Tree Model Confusion Matrix - Validation



The confusion matrix for the 600-tree random forests model (Figure 12) shows that the model correctly identified 430 potential fraud cases and 2,780 good, or unsuspecting, cases. It misclassified 570 potential fraud cases as good (missed fraud) and incorrectly flagged 220 good cases as potential fraud (false alarms). Overall accuracy is, as was noted above, 80.25%.

However, because the no information rate is 0.75 (what you would get by always predicting a non-suspicious transaction), accuracy alone can be misleading.

The positive predictive value) is 0.6615, so when the model flags potential fraud, it is correct about two-thirds of the time, while the negative predictive value is 0.8299, meaning predictions of good are correct about 83 percent of the time. The model is stronger on the non-fraud side but notably weaker at detecting fraud, which may be a concern depending on how costly missed fraud is relative to false alarms.

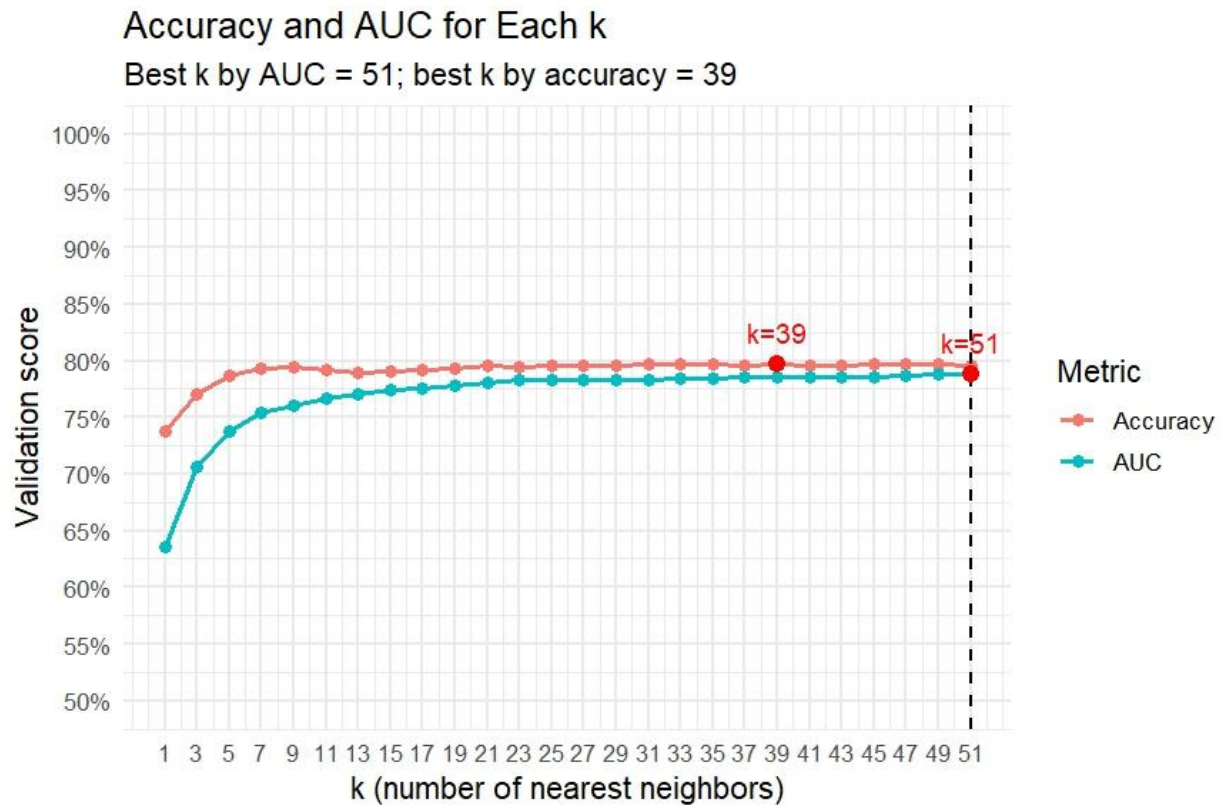
Table 12: Random Forest (600 trees) Validation Set: Accuracy and Statistical Significance

Metric	Value
Accuracy	0.8025
Accuracy 95% CI Lower	0.7898
Accuracy 95% CI Upper	0.8147
P-Value (Accuracy > NIR)	0.0000

It is also important to look at Table 12, which shows that the Random Forest (600 Trees) Model has a p-value of virtually with a 95% confidence interval over (0.7898, 0.8147). This indicates that although the difference may be small, the chose random forests model will almost always perform a little bit better than just labeling every transaction as non-suspicious would.

7. Analysis of K-Nearest Neighbors (KNN)

Figure 13: Accuracy and AUC for Each k



As Figure 13 indicates, across the validation set, the KNN model improves in both accuracy and AUC as k increases from 1 into the mid-to-high range, and then performance levels off. With very small k values the model is clearly weaker. For instance, at k = 1 the validation accuracy is 0.7380 and the AUC is 0.6360. As k increases, accuracy rises quickly into the upper 0.78 to low 0.79 range by about k = 5 to 9, while AUC increases into the 0.75 to 0.76 range over the same range of k. This pattern suggests that using more neighbors reduces overfitting and produces a more stable classifier than the highly local behavior at k = 1.

Continuing with Figure 14, after approximately k = 21, the gains nearly flatten and any gains are quite small. This indicates there are diminishing returns from continuing to increase k. In this region, validation accuracy stays tightly clustered around roughly 0.796 to 0.798, and

AUC continues to increase only gradually. The highest validation accuracy occurs at $k = 39$ (accuracy = 0.7975). However, several nearby k values are nearly tied in accuracy, showing that the model's performance is not very sensitive to the exact choice of k once it reaches this range.

With regards to AUC, as seen in Figure 14, the best-performing model occurs at $k = 51$ with a validation AUC of 0.78765. Although $k = 51$ does not produce the single highest accuracy, its accuracy (0.7960) is only slightly lower than the maximum. This difference is small enough that it likely reflects normal variation rather than any meaningful change in performance. There is no clear best model, $k = 51$ is the preferred KNN setting because it provides the strongest discrimination while maintaining essentially the same accuracy as $k = 39$.

Figure 14: KNN ($k = 51$) Confusion Matrix - Validation

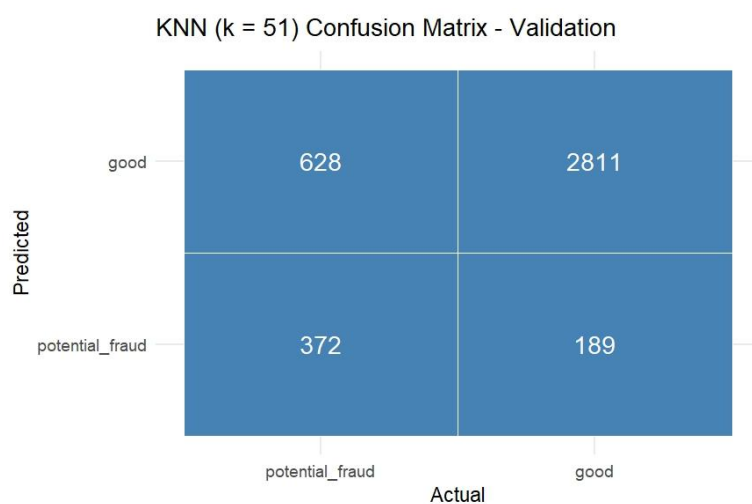


Figure 14 shows a confusion matrix for the KNN model when $k = 51$. The overall accuracy is 0.7958, meaning the model correctly classifies about 79.6 % of the validation observations. Since one could achieve 75% accuracy by just assuming that every transaction is not suspicious, this is not particularly impressive.

Table 13: KNN (k = 51) Validation Set: Accuracy and Statistical Significance

Metric	Value
Accuracy	0.7958
Accuracy 95% CI Lower	0.7829
Accuracy 95% CI Upper	0.8081
P-Value (Accuracy > NIR)	0.0000

However, as Table 13 shows, there is a very small p-value of virtually with a 95% confidence interval that is (0.7829, 0.8081). This small p-value indicates that although, in terms of correctly labeling transactions, the difference between using this model and labeling everything non-suspicious, it is virtually certain that this model is an improvement over this other method.

8. Comparing All Three Models when Run on Testing Data

Table 14: Test Set Model Comparison: Accuracy and AUC

Model	Accuracy	AUC
Decision Tree (thr=0.356)	0.7622	0.6081
Random Forest (ntree_600, thr=0.5)	0.8090	0.8054
KNN (k=51)	0.7975	0.7896

The three selected models were then run on the testing data and Accuracy and AUC scores can for each model can be seen in Table 14. When compared to each other, the random forest model performs the best. It achieves an accuracy of 0.8090 and an AUC of 0.8054, which indicates it not only makes the highest proportion of correct classifications but also has the strongest ability to separate potential fraud from good cases across different thresholds. The KNN model with k = 51 is a close second, with an accuracy of 0.7975 and an AUC of 0.7896.

The KNN model is thus fairly competitive, but it is consistently behind the random forest on both metrics. The decision tree performs the weakest, with an accuracy of 0.7622 and an AUC of 0.6081. While its accuracy is still above the 0.75 baseline that would come from always predicting a non-suspicious transaction, its AUC suggests it provides the smallest improvement over simply labeling every transaction as non-suspicious of any of the three models. Based on these results, the random forest model should be selected as it has both the highest accuracy and the highest AUC score.

9. Looking at the Superior Model

Table 15: Random Forest Model Variable Importance

Variable	potential_fraud	good	MeanDecreaseAccuracy	MeanDecreaseGini
housing_status	168.893	123.706	197.359	1532.215
credit_risk_score	33.713	65.788	76.453	1323.762
intended_balcon_amount	37.772	35.614	54.992	1172.226
days_since_request	9.757	17.489	20.767	1091.075
session_length_in_minutes	5.273	8.484	10.013	1059.471
velocity_4w	-10.740	67.094	62.466	1037.910
velocity_24h	-11.327	54.050	42.075	1015.035
velocity_6h	0.437	32.116	29.201	1007.099
income	55.263	33.924	58.932	587.450
proposed_credit_limit	35.102	70.484	84.571	581.170
customer_age	50.858	42.230	63.283	555.044
month	-9.201	60.898	57.321	405.134
payment_type	23.208	26.901	35.846	309.031
employment_status	33.163	10.976	28.393	220.353
foreign_request	6.799	12.883	14.779	64.340
source	-0.598	4.000	3.338	21.359

Table 15 has information about the impact of dependent variables on the overall random forest model. The four metrics displayed, potential_fraud, good, MeanDecreaseAccuracy, and MeanDecreaseGini, all measure the impact of the variable on the overall model. The information

in the columns labeled `potential_fraud` and `good` are class-specific importance measures that indicate how much each predictor helps the model correctly classify transactions. In this context, class-specific means the importance is measured separately for each outcome category rather than being combined into a single overall number, so it indicates how much a variable helps identify `potential_fraud` cases versus how much it helps identify non-suspicious cases. These importance values are based on the idea that the model scrambles the values of one variable (while leaving the others unchanged) and then checks how much prediction performance drops. The `potential_fraud` column thus reflects how strongly that variable contributes to identifying potential fraud cases. As one might expect, the `good` column reflects how strongly it contributes to correctly identifying good cases. Larger positive values mean the variable is more helpful for that class, values near zero mean it does not change performance much, and negative values can occur when the variable is mostly noise or slightly confusing for the model for that class where scrambling the variable can sometimes lead to a small improvement just by chance.

`MeanDecreaseAccuracy` uses the same concept and measures how much the overall accuracy of a model decreases when the variable is scrambled. `MeanDecreaseGini` is different as it is based on how often and how effectively a variable is used to split the data inside the trees, with larger values meaning the variable tends to produce cleaner splits that separate the classes more effectively. It should finally be noted that all of these metrics are measures of correlation, not causation.

Looking at the `potential_fraud` importance scores, `housing_status` (168.893) has the strongest impact on identifying potential fraud by far, with a much larger score than any other variable, which suggests the model relies heavily on housing status information when separating fraud from non-fraud cases. After `housing_status`, the next most influential predictors for

potential fraud are income (55.263) and customer_age (50.858). This indicates that customer-level characteristics play a meaningful role in the model's fraud detection. A second group of variables with moderate positive impact includes intended_balcon_amount (37.772), proposed_credit_limit (35.102), credit_risk_score (33.713), and employment_status (33.163), which suggests that the model also uses financial and credit-related context to flag higher-risk cases. Several variables have smaller but still positive potential_fraud contributions, such as payment_type (23.208), days_since_request (9.757), foreign_request (6.799), and session_length_in_minutes (5.273), implying that transaction context and timing provide additional signal but are not as dominant as the most significant factors. Finally, a few predictors show negative potential_fraud importance (velocity_4w (-10.740), velocity_24h (-11.327), month (-9.201), and source (-0.598)), meaning that in this model they do not consistently improve fraud-case classification and may be weak signals or overlapping with other variables.

Looking at the good importance scores, housing_status (123.706) still has the strongest impact on correctly identifying good (non-fraud) cases, which is consistent with the potential_fraud results where it was also the dominant driver (168.893), although its influence is somewhat smaller for good than it is for potential fraud. Unlike the potential_fraud column where income and customer_age were the next biggest contributors, the good column shows that credit_risk_score (65.788) and proposed_credit_limit (70.484) play a larger role in correctly classifying good cases. This suggests these variables are especially useful for avoiding false positives and confirming legitimate transactions. The velocity variables also stand out for how they contrast with the potential_fraud results. Specifically, velocity_4w (67.094) and velocity_24h (54.050) are strongly positive for good even as they were negative for potential_fraud (-10.740 and -11.327, respectively). This indicates they are more helpful for

recognizing non-fraud patterns than for catching fraud cases, at least in this model. Additionally, month (60.898) is strongly positive for good while it was negative for potential_fraud (-9.201), and velocity_6h (32.116) has a much larger positive role for good than for potential_fraud (0.437). Several predictors contribute moderately to good classification, including customer_age (42.230), intended_balcon_amount (35.614), income (33.924), and payment_type (26.901), while days_since_request (17.489), foreign_request (12.883), employment_status (10.976), and session_length_in_minutes (8.484) provide smaller supporting signal. Lastly, source (4.000) remains relatively low for good. This is similar to how it was near 0 and slightly negative for potential_fraud (-0.598) and suggests it has limited influence on classification overall compared to the other predictors.

Given that MeanDecreaseAccuracy is something of a composite of potential_fraud and good, it should come as no surprise that housing_status (197.359) has by far the largest impact on overall predictive performance. The next most influential variable according to this metric is proposed_credit_limit (84.571), which aligns more closely with the good results where it was very strong (70.484) than with the potential_fraud results where it was moderate (35.102), suggesting proposed_credit_limit is especially important for maintaining overall correctness by helping the model recognize non-fraud cases. credit_risk_score (76.453) and customer_age (63.283) also contribute heavily to overall accuracy, which fits with credit_risk_score being much stronger for good (65.788) than for potential_fraud (33.713), and customer_age being important for both classes (50.858 for potential_fraud and 42.230 for good). There are several other predictors that show a substantial overall impact, including velocity_4w (62.466), income (58.932), month (57.321), and intended_balcon_amount (54.992). Even though some of these had negative potential_fraud class-specific values (velocity_4w at -10.740 and month at -9.201),

indicating that while they may not consistently improve fraud-case identification on their own, they still help the model's total accuracy. A middle tier of predictors includes velocity_24h (42.075), payment_type (35.846), velocity_6h (29.201), and employment_status (28.393), while days_since_request (20.767), foreign_request (14.779), and session_length_in_minutes (10.013) have smaller but still positive effects on overall performance. Finally, source (3.338) has the smallest MeanDecreaseAccuracy, which is consistent with it being near 0 in both class-specific columns. This suggests it contributes relatively little to the model's predictive power compared to the other variables.

Looking at MeanDecreaseGini, housing_status (1532.215) again stands out as the most influential variable by a wide margin. This means it is used frequently and produces strong splits that separate the data into cleaner fraud versus non-fraud groupings and is consistent with earlier results where housing_status was also the top driver for both potential_fraud (168.893) and good (123.706) and had the largest MeanDecreaseAccuracy (197.359). The next strongest variables by MeanDecreaseGini are credit_risk_score (1323.762) and intended_balcon_amount (1172.226). This indicates that the random forest model relies heavily on credit risk information and requested/intended amount information when building decision rules; this also matches the pattern that credit_risk_score is especially important for correctly identifying good cases (65.788) while still contributing to potential_fraud detection (33.713) while intended_balcon_amount contributes relatively evenly to both classes (37.772 for potential_fraud and 35.614 for good). A large set of behavioral and timing variables also plays a major role in how the trees split the data, including days_since_request (1091.075), session_length_in_minutes (1059.471), velocity_4w (1037.910), velocity_24h (1015.035), and velocity_6h (1007.099), which is consistent with the good column where the velocity measures are strongly positive

(67.094, 54.050, and 32.116, respectively) even though two of them are negative for potential_fraud (-10.740 and -11.327, respectively) This indicates they help the forest carve out patterns for legitimate activity more than they directly improve fraud-case classification. In contrast, more traditional customer and account attributes such as income (587.450), proposed_credit_limit (581.170), and customer_age (555.044) still matter but appear less central to the forest's splitting strategy than the top credit, amount, and velocity-related variables. This is true even though they can show strong class-specific importance for potential_fraud (income at 55.263 and customer_age at 50.858) and for good (proposed_credit_limit at 70.484). Finally, month (405.134), payment_type (309.031), and employment_status (220.353) have moderate influence on splitting, while foreign_request (64.340) and source (21.359) contribute relatively little compared to the rest of the variables.

10. Business and Managerial Implications

The most significant business and managerial implication of this study is that financial institutions can use machine learning to better predict what transactions should be reviewed for potential APP fraud, but also that financial institutions need to remember that their goal is to create a better experience for their customers, not merely to effectively identify transactions as suspicious and non-suspicious. In practical terms, the selected random forest model can be implemented as a decision-support capability that assigns a risk score to each customer-initiated payment request and supports prioritization of attention toward the cases most likely to involve APP fraud. The institution using this model would then be better able to allocate its (presumably limited) investigative and operational resources by focusing on transactions that are more likely to involve APP fraud. Over time, such an approach can contribute to reduced fraud losses, improved risk management outcomes, and more customer satisfaction.

From a tactical perspective, the model supports the establishment of a standardized framework for identifying and responding to suspicious payments across the channels where transactions are made (e.g., mobile banking, online banking, and branch-related interactions). A centralized scoring process can reduce variability in human decision-making, improve the documentation and auditability of fraud controls, and enhance reporting to risk and compliance stakeholders such as insurers. The model output can also be integrated into tiered control strategies, in which higher-risk transactions trigger proportionate interventions such as delayed processing pending confirmation or direct customer outreach. (The latter might actually be best done after some amount of time has passed and this might make it less likely that a victim of APP fraud is the presence of the fraudster.) This is particularly relevant in APP fraud, where the transaction is initiated and authorized by the customer; therefore, effective prevention often depends on implementing timely controls and warnings that interrupt manipulation before funds are transferred.

In addition, the variable-importance results provide insight that, rather than being limited to transaction flagging, can be used to inform broader prevention and customer-protection strategies. In this analysis, `housing_status` emerged as the strongest driver of the model's predictions, indicating that customer context variables, when they are combined with financial and behavioral signal, are highly informative in distinguishing higher-risk transactions from lower-risk ones. Accordingly, the bank could use these insights to design targeted, supportive interventions aimed at reducing customer vulnerability to scams. For example, when a payment request exhibits a risk profile consistent with elevated APP fraud likelihood, the bank could deploy enhanced in-app warnings, more explicit confirmation prompts, and immediate educational messaging regarding common APP scam tactics. These measures would emphasize

customer protection and fraud prevention while avoiding the inappropriate use of individual characteristics as standalone grounds for adverse actions.

Several limitations exist and should be recognized when interpreting and applying these findings. As was mentioned above, the dataset used in this project is synthetic and therefore may not fully represent the complexity of real customer behavior and the dataset was not meant to represent APP fraud, which further solidifies this problem. Model performance and variable relationships observed in this analysis may not generalize directly to a real-world banking environment. Furthermore, APP fraud patterns can change rapidly as criminals adapt to controls (Financial Conduct Authority, 2023), which means that any deployed model would require active monitoring and periodic retraining to remain effective.

Finally, because certain predictors may function as proxies for broader socioeconomic factors, a real-world implementation would require appropriate governance, including compliance review and fairness assessment, particularly if model outputs affect customer experience through increased friction, delays, or verification requirements. For these reasons, the model should be positioned something that informs proportionate fraud controls, rather than the only source of decision-making.

There are clear opportunities to enhance both the model and its managerial usefulness in a real banking context. The most meaningful improvement would be to train and validate the model using real institutional data with confirmed APP fraud outcomes, including investigation conclusions, customer-reported scam events, dispute outcomes, and associated loss amounts. In addition, the identification of additional variables that better reflect APP fraud mechanisms, and the inclusion of these variables in the model, could improve the bank's ability to identify high-risk payments while reducing unnecessary friction for legitimate transactions. Finally,

management should evaluate the solution not only on predictive performance, but also on business outcomes such as prevented losses as it could be that certain variables are more closely correlated to monetarily larger cases of APP fraud.

11. Conclusion

This paper set out to address the growing challenge of authorized push payment (APP) fraud in the financial sector, particularly as evolving technology and social engineering tactics make detection increasingly difficult. By leveraging a large synthetic dataset, it evaluated the ability of three machine learning techniques -decision trees, random forests, and K-Nearest Neighbors- to distinguish potentially fraudulent transactions. Our analysis found that the random forest approach offered the best balance of accuracy and discrimination, with customer and contextual variables such as housing status and credit information proving particularly influential. While these results indicate that data-driven approaches can support financial institutions in prioritizing high-risk cases, they must be interpreted in light of significant limitations such as this paper's use of synthetic, non-APP-specific data and the risk of rapidly changing fraud patterns that may outpace static analytical models. Ultimately, this study underscores both the potential and the ongoing challenges of using machine learning to combat APP fraud and should be understood as pointing to the need for regular model updates, strong governance, and careful attention to customer experience and fairness when such a model is used by a financial institution.

References

- Arrow, K. J. (1972). Gifts and Exchanges. *Philosophy & Public Affairs*, 1(4), 343–362.
- Arlot, S., & Celisse, A. (2010). A survey of cross-validation procedures for model selection. *Statist. Surv*, 4(none), 40–79. <https://doi.org/10.1214/09-SS054>
- Bao, Y. (2023). The impact of financial fraud on economic stability: An extensive economic analysis. *Journal of Internet Banking and Commerce*, 28(4), 1-3.
- Bertsimas, D., O’Hair, A.K., & Pulleyblank, W.R. (2016). *The Analytics Edge*. Dynamic Ideas LLC.
- Braithwaite, J. (2024). ‘Authorized Push Payment’ Bank Fraud: What Does an Effective Regulatory Response Look Like? *Journal of Financial Regulation*, 10(2), 174–193. <https://doi.org/10.1093/jfr/fjae006>
- Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>
- Financial Conduct Authority. (2023). Authorised push payment (APP) fraud – data and trends. <https://www.fca.org.uk/insight/authorised-push-payment-fraud-data-trends>
- Galoustian, G. (2025, April 15). Busted! Engineers Revolutionize Fraud Detection with Machine Learning. *Florida Atlantic University*. <https://www.fau.edu/newsdesk/articles/machine-learning-fraud-detection>
- Halabaku, E., & Bytyçi, E. (2024). Overfitting in Machine Learning: A Comparative Analysis of Decision Trees and Random Forests. *Intelligent Automation and Soft Computing*, 39(6), 987–1006. <https://doi.org/10.32604/iasc.2024.059429>

Hashemi-Pour, C., Yasar, K., & Laskowski, N. (2024, December 26). What is synthetic data?

Examples, use cases and benefits. *TechTarget*.

<https://www.techtarget.com/searchcio/definition/synthetic-data>

Jesus, S., Pombal, J., Alves, D., Cruz, A.F., Saleiro, P., Ribeiro, R.P., Gama, J., & Bizarro, P.

(2022). *Bank Account Fraud Dataset Suite (NeurIPS 2022)*. [Data set]. Kaggle.

<https://www.kaggle.com/datasets/sgpjesus/bank-account-fraud-dataset-neurips-2022>

Knack, S., & Keefer, P. (1997). Does Social Capital Have an Economic Payoff? A Cross-Country Investigation. *The Quarterly Journal of Economics*, 112(4), 1251–1288.

<http://www.jstor.org/stable/2951271>

Lalchand, S., Srinivas, V., Wadhwani, R., & Maggiore, B. (2025, October 9). Forecasting the rise of push payment scams—the fraud consumers are tricked into authorizing. *Deloitte*

Insights. <https://www.deloitte.com/us/en/insights/industry/financial-services/authorized-push-payment-fraud.html>

Nasdaq. (2024, January 16). Nasdaq releases first global financial crime report, measuring the scale and human impact of financial crime. *NASDAQ OMX's News Release Distribution Channel*. <https://www.nasdaq.com/press-release/nasdaq-releases-first-global-financial-crime-report-measuring-the-scale-and-human>

Ngan, J. (2025). “The view from below”: Resistance and change in Authorised Push Payment Fraud. *Journal of Economic Criminology*, 9, Article 100166.

<https://doi.org/10.1016/j.jeconc.2025.100166>

Pavlou, P. A. (2003). Consumer Acceptance of Electronic Commerce: Integrating Trust and Risk with the Technology Acceptance Model. *International Journal of Electronic Commerce*, 7(3), 101–134. <http://www.jstor.org/stable/27751067>

- Payment Systems Regulator. (2026, February). APP scam performance data: 2024 (covering January–October 2024). <https://www.psr.org.uk/media/b50p3mlm/psr-2024-app-scam-performance-data-february-2026.pdf>
- Pestov, V. (2013). Is the k-NN classifier in high dimensions affected by the curse of dimensionality? *Computers & Mathematics with Applications (1987)*, 65(10), 1427–1437. <https://doi.org/10.1016/j.camwa.2012.09.011>
- Raghunathan, T., Reiter, J., & Rubin, D. (2003). Multiple Imputation for Statistical Disclosure Limitation. *Journal of Official Statistics*, 19(1), 1-16.
- Reiter, J.P. (2005). Using CART to generate partially synthetic public use microdata. *Journal of Official Statistics*, 21(3), 441-462.
- Sapienza, P., & Zingales, L. (2012). A Trust Crisis. *International Review of Finance*, 12(2), 123–131. <https://doi.org/10.1111/j.1468-2443.2012.01152.x>
- ScienceDirect. (n.d.). K-Nearest neighbor classifier. <https://www.sciencedirect.com/topics/computer-science/k-nearest-neighbor-classifier>
- Shaw, V. (2024, August 1). APP fraud reimbursement still largely depends on who victims bank with. *The Independent*. <https://www.the-independent.com/money/app-fraud-reimbursement-still-largely-depends-on-who-victims-bank-with-b2589397.html>
- Shum, N. (2025). Authorised push payments fraud – theorising a loss allocation model. *Journal of Law and Jurisprudence*, 14(1), 43-69. <https://doi.org/10.14324/111.444.2052-1871.1988>
- Snoke, J., Raab, G. M., Nowok, B., Dibben, C., & Slavkovic, A. (2018). General and Specific Utility Measures for Synthetic Data. *Journal of the Royal Statistical Society Series A: Statistics in Society*, 181(3), 663–688. <https://doi.org/10.1111/rssa.12358>

Stadler, T., Oprisanu, B., & Troncoso, C. (2022). Synthetic Data -- Anonymisation Groundhog Day. *arXiv.Org*. <https://arxiv.org/abs/2011.07018>

Standard Chartered. (n.d.). APP reimbursement policy. <https://www.sc.com/uk/app-reimbursement-policy/>

Vona, L. W. (2017). *Fraud data analytics methodology: the fraud scenario approach to uncovering fraud in core business systems* (1st ed.). John Wiley & Sons.

Yang, X. (2025). Financial fraud identification and economic impact analysis based on machine vision technology. *Journal of Computational Methods in Sciences and Engineering*, 25(6), 5544–5559. <https://doi.org/10.1177/14727978251346036>